

Volume Segmantics

JOSS 10.21105/joss.04691

tests

no status

Build and Release

no status

Volume Segmantics is a PyTorch Deep-learning Segmentation Toolkit providing a simple command-line interface and API that allows researchers to quickly train a variety of 2D PyTorch segmentation models (e.g. U-Net, U-Net++, FPN, DeepLabV3+) on their 3D datasets. These models use pre-trained encoders, enabling fast training solutions on a range of biological datasets where it has been optimised for minimal data input; a single dataset in between 128 and 512 cubic pixels, is able to produce efficient and accurate predictions on much larger comparable images. Great success has been seen even using small computing power, however with increased capability, and using the toolkits advanced functionality, larger and more complex datasets can now be processed with a reduced timeframe.

Given a 3D image volume and corresponding dense labels (the segmentation), a 2D prediction model is trained on image slices taken along the x, y, and z axes, where image augmentations are used to expand the size of the training datasets. Subsequently, the library enable using these trained models to segment larger 3D datasets, automatically merging predictions made in orthogonal planes and rotations to reduce artifacts that may result from predicting 3D segmentation using a 2D network.

This work utilises the abilities afforded by the excellent [segmentation-models-pytorch](#) library in combination with augmentations made available via [Albumentations](#) and from the [MONAI](#) library. Also the metrics and loss functions used make use of the hard work done by Adrian Wolny in his [pytorch-3dunet](#) repository.

Requirements

A machine capable of running CUDA enabled with a recent version of Pytorch (2.5 or greater is recommended); this generally means a reasonably modern NVIDIA GPU. The exact requirements differ according to operating system. For example, on Windows you will need Visual Studio Build Tools as well as CUDA Toolkit installed; see [the CUDA docs](#) for more details.

Installation

Conda/Virtual_env (PyPI)

The latest published release may be installed from the Python Package Index in a new conda environment or virtual_env with python (ideally >= version 3.10) and pip. For more information, documentation for [conda](#) and [pip](#) can be found at these sites respectively. Simply activate your new environment and install;

```
pip install volume-segmantics
```

If you find a CUDA-enabled build of PyTorch is not being installed by pip (this particularity seems to be an issue on Windows), you can try this adaption;

```
pip install volume-segmantics --extra-index-url https://download.pytorch.org/whl
```

Docker/Apptainer Container (quay.io)

A container image with the latest published release is available on the Rosalind Franklin's quay.io instance. You can pull and run this using Apptainer:

```
apptainer run --nv docker://quay.io/rosalindfranklininstitute/volume-segmentics
```

Or Docker:

```
docker run \  
  --gpus all \  
  --ipc=host \  
  -v /path/to/data:/data  
  quay.io/rosalindfranklininstitute/volume-segmentics
```

Note `--ipc=host` grants the container access to host shared memory, since the default allocation in Docker (64M) may be insufficient. More details can be found on the [Volume EM Container documentation](#).

Configuration and Command line use

After installation, two new commands will be available from your terminal whilst your environment is activated; `model-train-2d` and `model-predict-2d`.

These commands require access to YAML files settings stored in the VolSeg install. These need to be located in a directory named `volseg-settings` within the directory where you are running the commands. The Default settings files can be copied from [here](#) if changes to them are made and you need a fresh start.

The file `2d_model_train_settings.yaml` can be edited in order to change training parameters such as image size, model architecture, learning rate, loss functions, evaluation metrics and also more advanced training settings.

The file `2d_model_predict_settings.yaml` can be edited to change mirrored training parameters (that must remain consistent) and the prediction "quality" e.g "low", "medium" and "high" quality.

Further information regarding the **Default and more specific Settings** can be found detailed in the [Settings Guide](#) documentation.

Check the normalisation settings carefully for your intended use; different datasets may require different choices from the default; double-check that the prediction normalisation settings match those used for training.

For training a 2D model on a 3D image volume and corresponding label

Run the following command. Input files can be in HDF5 or multi-page TIFF format or MRC format. The `--data` and `--labels` arguments define the training image and label files used within the model training, and describe a single image and label match as an **Image-Label Pair**. The relative size of each X-Y-Z measurement does

not have to be equal (not a perfect square), however the *Image-Label Pair* must be the same size spatially and comparable in 3 dimensions, fitting into each other without gaps to be successful.

```
model-train-2d --data path_to_image_data --labels path_to_segmentation_labels
```

Paths to multiple Image-Label Pairs can also be added after the `--data` and `--labels` flags respectively; keep in mind these must be chronological per flag. A output model will be trained according to the training settings YAML, saved to your working directory and contain 4 files;

- 1] **.pytorch** model file; the segmentation model created by the train command that can be used to predict on other images,
- 2] **model_loss-Plot.png**; a graph showing the training and validation loss over the course of the training epochs (a *graphical* representation of the model's success),
- 3] **model_prediction_image.png**; showing specific slices of the image and labels used within the model, and the resultant prediction test outcomes for that slice (a *visual* representation model's success),
- and 4] **stats.csv**; a record of the training loss, validation loss and evaluation score per epoch (comparable to a DiceScore (1.0 equals 100% accuracy); accuracy of the test prediction generated versus the original label)(a numerical representation model's success)..

For 3D segmentation prediction using a 2D model

Run the following command. Input image files can be in HDF5 or multi-page TIFF format or MRC format, though the model file must be in .pytorch format.

```
model-predict-2d path_to_model_file path_to_prediction_data
```

The input data will be segmented using the input model following the settings specified in the prediction setting YAML. Depending on your set OUTPUT_FORMAT in *config.py*, set to "tif" or "hdf" or "mrc", a relative label file containing the segmented volume will be saved to your working directory.

Training features

Volume Segmantics supports training using a variety of U-net encoder-decoder architectures and encoder choices, including transformer-based models such as DINO. It allows purely axial or tri-planar/multi-axis prediction as well as supporting multiple decoder and multiple head architectures and multi-task training through a configurable `pipeline.yaml`.

Volume Segmantics supports **2.5D slicing**, which creates multi-channel images from adjacent slices in the volume. This feature can be enabled by setting `use_2_5d_slicing: True` in the training settings file. This approach provides the model with spatial context from adjacent slices. The encoder adjusts to use the number of input channels specified by the num_slices parameter in training settings YAML (only when 2.5D slicing is enabled). When using 2.5D slicing for training, you must also enable 2.5D prediction for inference by setting `use_2_5d_prediction: True` in your prediction settings YAML file and set the same num_slices parameter there.

Further information regarding the **specific utilities** and **additional or more advanced training and prediction options** can be found detailed in the [Functionality](#) documentation.

Tutorial using example data

A **tutorial** using the toolkits default settings, and containng detailed instructions for initial environment setup and settings YAML configuration, is available [here](#); it provides a walk-through of how to segment blood vessels from synchrotron X-ray micro-CT data collected on a sample of human placental tissue.

Contributing

We welcome contributions from the community. Please take a look at out [contribution guidelines](#) for more information.

Citation

If you use this package for your research, please cite:

King O.N.F, Bellos, D. and Basham, M. (2022). Volume Segmantics: A Python Package for Semantic Segmentation of Volumetric Data Using Pre-trained PyTorch Deep Learning Models. Journal of Open Source Software, 7(78), 4691. doi: 10.21105/joss.04691

```
@article{King2022,
  doi = {10.21105/joss.04691},
  url = {https://doi.org/10.21105/joss.04691},
  year = {2022},
  publisher = {The Open Journal},
  volume = {7},
  number = {78},
  pages = {4691},
  author = {Oliver N. F. King and Dimitrios Bellos and Mark Basham},
  title = {Volume Segmantics: A Python Package for Semantic Segmentation of Volumetric Data Using Pre-trained PyTorch Deep Learning Models},
  journal = {Journal of Open Source Software} }
```

References

Albumentations

Buslaev, A., Iglovikov, V.I., Khvedchenya, E., Parinov, A., Druzhinin, M., and Kalinin, A.A. (2020). Albumentations: Fast and Flexible Image Augmentations. Information 11. <https://doi.org/10.3390/info11020125>.

MONAI

Cardoso, M. Jorge, Wenqi Li, Richard Brown, Nic Ma, Eric Kerfoot, Yiheng Wang, Benjamin Murrey et al. (2022) "Monai: An open-source framework for deep learning in healthcare." [arXiv preprint arXiv:2211.02701](#).

Segmentation Models PyTorch

Yakubovskiy, P. (2020). Segmentation Models Pytorch. [GitHub](#).

PyTorch-3dUnet

Wolny, A., Cerrone, L., Vijayan, A., Tofanelli, R., Barro, A.V., Louveaux, M., Wenzl, C., Strauss, S., Wilson-Sánchez, D., Lymbouridou, R., et al. (2020). Accurate and versatile 3D segmentation of plant tissues at cellular resolution. *ELife* 9, e57613. <https://doi.org/10.7554/eLife.57613>.